

L22 - Repetition statements 2 - while do-while

Department of Computer Science
University of Pretoria

30 April 2024

Loop: a control structure that causes the execution of a statement or block of statements to be repeated.

General format of the **while** loop:

```
while (expression)
    statement;

OR:
while (expression)
{
    statement1;
    statement2;
    .....
    statementn;
}
```

while loop is executed as **long** as expression has a **true** value

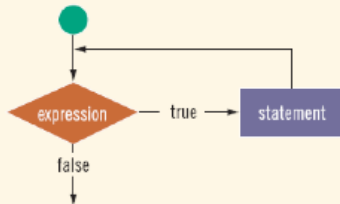


FIGURE 5-1 `while` loop

Figure: While loop flow chart

Infinite loop: continues to execute endlessly
Avoided by including statements in loop body

Listing 1: C++ program segment

Consider the following program segment:

```
#include <iostream>
using namespace std;
int main()
{
    int i= 0;

    while (i <= 20)
    {
        cout << i <<" -" ;
        i= i + 5;
    }
    cout << endl;

    return 0;
}
//Output:
0 5 10 15 20
```

Listing 2: C++ program segment that never executes

```
#include <iostream>
using namespace std;

int main()
{
    int i= 20;

    while (i < 20)
    {
        cout << i <<" -";
        i= i + 5;
    }
    cout << endl;
    return 0;
}
```

Output:

No Output, because $i = 20$, the expression $i < 20$ evaluates **false**

Listing 3: While loop example

What is the output of the following code:

```
#include <iostream>
using namespace std;

int main()
{
    int number    =    1;

    while ( number    <=    5 )
    {
        cout    <<    " Hello\n";
        number++;
    }

    cout<< " thats - all - \n";

    return 0;
}
//
```

This section we discuss Infinite loops:

An infinite loop is a loop that executes indefinitely.
Human intervention is used to terminate execution.

Listing 4: Infinite loops

This **while** loop is NOT an infinite loop. Why?

This is NOT an infinite loop

```
int number = 1;

while (number <= 5)
{
    cout << " Hello\n";
    number++;
}
```

BUT **this while** loop is an infinite loop. Why?

This is an infinite loop

```
int number = 1;

while (number <= 5)
{
    cout << "Hello\n";
}
```


Sentinel-Controlled while Loops:

Sentinel variable is tested in the condition
and loop ends when sentinel is encountered

```
cin >> variable;           //initialize the loop control variable

while (variable != sentinel) //test the loop control variable
{
    .
    .
    .
    cin >> variable;        //update the loop control variable
    .
    .
    .
}
```

Flag-Controlled while Loops:

A flag-controlled while loop uses a bool variable to control the loop.

The flag-controlled while loop takes the form:

```
found = false;           //initialize the loop control variable
while (!found)           //test the loop control variable
{
    .
    .
    .
    if (expression)
        found = true; //update the loop control variable
    .
    .
}
```

Listing 5: While loops with operator

```
#include <iostream>
using namespace std;

int main() {
    int number, sum = 0;

    cout << "Enter positive numbers (enter 0 to stop):" << endl;

    while (cin >> number && number > 0) {
        sum += number;
    }

    cout << "Sum of positive numbers entered: -" << sum << endl;

    return 0;
}
```

General form of a do...while:

```
do  
    statement  
while (expression);
```

The statement executes first, and then the expression is evaluated

To avoid an infinite loop, body must contain a statement that makes the expression false

The statement can be simple or compound

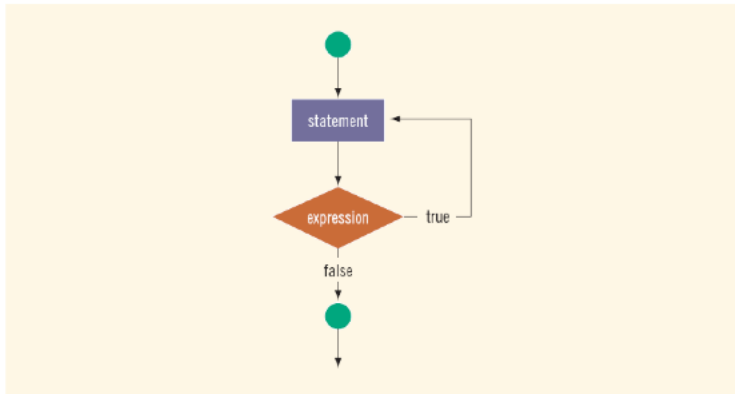


FIGURE 5-3 do...while loop

Figure: do... while flow chart

Listing 6: do...while example

```
#include <iostream>
using namespace std;

int main() {
    int i = 0;
    do
    {
        cout << i <<" -" ;
        i = i+5;
    }
    while (i <=20);
    return 0;
}
//Output
0 5 10 15 20
```

do-while loop

- always executes at least once
- execution continues as long as expression is true,
- stops repetition when expression becomes false

Very similar, but... One important difference
Issue is "WHEN" boolean expression is
checked

- while: checks BEFORE body is executed
- do-while: checked AFTER body is
executed

After this difference, they're essentially
identical! while is more common, due to it's
ultimate "flexibility"


```
//This program averages 3 test scores. It repeats as
//many times as the user wishes
#include <iostream>
using namespace std;
int main()
{
    int score1, score2, score3;
    double average;
    char again;

    do {
        // Get the three scores.
        cout << "Enter 3 scores and I will average them: ";
        cin >> score1 >> score2 >> score3;
        // Calculate and display the average.
        average = (score1 + score2 + score3) / 3.0;
        cout << "The average is " << average << ".\n";
        // Does the user want to average another set?
        cout << "Do you want to average another set? (Y/N): ";
        cin >> again;
    } while (again == 'Y' || again == 'y');
    return 0;}
```

Going back to **this** code:
what is the output of **this**:

```
#include <iostream>
```

```
int main() {  
    // Initialize 'i' for regular increment  
    // and 'k' for staggered increment  
    for (int i = 0, k = 0; i < 10; i++, k += (i % 2 == 0 ? 1 : 0))  
        std::cout << "i=-" << i << ",-k=-" << k << std::endl;  
}  
  
    return 0;}
```

Output:

```
i = 0, k = 0  
i = 1, k = 0  
i = 2, k = 1  
i = 3, k = 1  
i = 4, k = 2  
i = 5, k = 2  
i = 6, k = 3  
i = 7, k = 3
```

Choosing the Right Looping Structure

```
i = 8, k = 4
```

```
i = 9, k = 4
```

The **do-while** code for the previous code:

```
#include <iostream>
using namespace std;

int main() {
    int i = 0, k = 0;

    do {
        cout << "i=-" << i << ", -k=-" << k << endl;
        i++;
        k += (i % 2 == 0 ? 1 : 0);
    } while (i < 10);

    return 0;
}
//same output as the previous code
//
```

All three loops have their place in C++

- If you know or can determine in advance the number of repetitions needed, the for loop is the correct choice
- If you do not know and cannot determine in advance the number of repetitions needed, and it could be zero, use a while loop

- If you do not know and cannot determine in advance the number of repetitions needed, and it is at least one, use a do...while loop

Class quiz:

Write a C++ code using a do while loop that ask the user to enter number between 1 to 10 and **checks if its even or odd numbers**, define any loop condition of your choice.

```
//Class quiz solution
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int number;
```

```
do {
```

```
    cout << "Enter a number between 1 and 10: ";
```

```
    cin >> number;
```

```
if (number < 1 || number > 10) {
```

```
    cout << "Invalid input! Please enter a number between 1 and 10.\n";
```

```
    } else {
```

```
        if (number % 2 == 0) {
```

```
            cout << "The number-" << number << "- is even." << endl;
```

```
        } else {
```

```
            cout << "The number-" << number << "- is odd." << endl;
```

```
        }
```

```
    }
```

```
    } while (number < 1 || number > 10);
```

```
    return 0;  
}
```